

Cross-Residual Graph Neural Networks for Biomolecular Graph Classification: An Empirical Study of Residual Routing

Jingchao Wang¹, Wei Huang¹, Xuelin Hu², and Xiaoqin Fu²

¹School of Computer Science, Zhongyuan University of Technology, Zhengzhou 450000, China

²School of Railway Communication and Signaling Engineering, Liuzhou Railway Vocational Technical College, Liuzhou, Guangxi, China

Corresponding author:

Xuelin Hu¹

Email address: huxuelinai@gmail.com

ABSTRACT

Background. Graph neural networks are widely used for graph classification on proteins and molecules, but deeper message passing can blur discriminative local structure, and standard skip connections do not test whether cross-branch historical exchange offers a better trade-off than ordinary residual reuse. This issue is especially relevant in biomolecular graph classification, where useful evidence may depend on both local motifs and graph-level context.

Methods. We study Cross-Residual Graph Neural Networks (CR-GNN), a controlled model family that compares plain stacking, node-level residual reuse, node-level cross-residual exchange, graph-level residual propagation, and graph-level cross-summary exchange under shared graph-classification backbones. The final benchmark uses six datasets, nine methods, four message-passing operators, and stratified five-fold evaluation. We further organize targeted experiments on gate control, residual routing modes, residual-parameter sweeps, and exploratory sensitivity scans.

Results. The final results show that residual reuse remains the stronger default family across the biomolecular benchmark package, while cross-residual routing usually improves over plain stacking and yields selective gains on specific datasets. The strongest graph-level residual line remains best on several core settings, but residual routing design materially affects the outcome: mild sparse routing is often competitive and is the strongest family on three of four representative residual-routing targets.

Conclusions. CR-GNN is best interpreted as a biomolecular graph-classification design space rather than a universally dominant architecture. The empirical evidence supports a constrained conclusion: cross-residual interaction is useful but conditional, and residual routing strategy is an important determinant of performance.

INTRODUCTION

Graph neural networks (GNNs) are now a standard approach for learning from molecular and protein graphs because they encode local neighborhood structure together with higher-order relational context [10, 6, 8]. In biomolecular graph classification, however, deeper message passing is not automatically beneficial. Repeated propagation can smooth away informative intermediate states, and conventional skip connections only test same-path reuse rather than richer alternatives for preserving historical information.

This paper studies Cross-Residual Graph Neural Networks (CR-GNN), a controlled model family that compares plain stacking, node-level residual reuse, node-level cross-residual exchange, graph-level residual propagation, and graph-level cross-summary exchange under shared graph-classification backbones. The central question is deliberately narrower than a generic model race: when does cross-residual interaction help on biomolecular graph classification, when does residual reuse remain the stronger default, and how does residual routing design change that answer?

The work is organized as a biomolecular graph-classification methods study rather than as a direct biological discovery paper. The main evidence comes from a protein-oriented benchmark core built around

47 PROTEINS, DD, and ENZYMES, together with supplementary molecular robustness datasets. This
48 design keeps the paper close to biologically meaningful graph settings while maintaining a reproducible
49 benchmark scope. The paper first motivates the architectural question, then presents the methodological
50 setting, the benchmark results, the targeted gate and routing ablations, and the supporting sensitivity
51 studies.

52 **Contributions**

53 The contributions of this work are summarized as follows:

- 54 • We formalize CR-GNN as a compact architecture family that compares five graph-classification vari-
55 ants under one controlled framework: PlainGNN, NodeResGNN, NodeCrossGNN, GraphResGNN,
56 and GraphCrossGNN.
- 57 • We extend the study beyond architecture comparison by introducing residual-routing analysis
58 through learnable, top-k, and sparse routing modes.
- 59 • We organize the final empirical study around a unified $\vee 3$ benchmark with six datasets, nine
60 methods, four operators, and stratified five-fold evaluation, supplemented by targeted gate and
61 routing ablations.
- 62 • We show that cross-residual interaction is useful but conditional: residual reuse remains the stronger
63 default family, while mild sparse routing is often advantageous on selected cross-oriented settings.

64 **BACKGROUND**

65 Graph classification on proteins and molecules has become a standard use case for graph neural networks
66 because graph structure preserves both local biochemical motifs and broader relational organization. In
67 biomolecular settings, this dual dependence matters: a graph label may reflect coordinated substructures
68 rather than one dominant neighborhood pattern, so useful prediction signals can be distributed across
69 depth, across branches, and across graph-level summaries. The background below therefore moves from
70 the application setting to the mechanism setting that motivates CR-GNN.

71 **Graph neural networks for biomolecular graph classification**

72 Graph-based learning is now routinely used for protein and molecular tasks in which the target depends
73 on structured relational information rather than on independent tabular descriptors alone [10, 6, 8]. In
74 biomolecular applications, graph neural networks have already been used for structure-aware protein
75 function prediction, protein interface modeling, and related molecular learning problems where the
76 relation among substructures is as important as the substructures themselves [7, 18, 15]. Benchmark
77 suites such as PROTEINS, DD, ENZYMES, MUTAG, AIDS, and Mutagenicity remain widely used
78 because they expose differences in graph size, sparsity, and classification difficulty while retaining direct
79 biological or molecular interpretation [4, 13]. In the present study, these datasets are used as a reproducible
80 biomolecular graph-classification package rather than as a proxy for downstream biological discovery.

81 **Residual reuse and deep message passing**

82 Deeper message passing can enlarge receptive fields, but it also increases the risk of representation
83 degradation. Over-smoothing and related depth effects can suppress informative intermediate states,
84 especially when repeated propagation progressively attenuates local variation [12, 14, 3]. Residual
85 connections, normalization, and regularization alleviate this problem [9, 1, 19, 2, 16], but most residual
86 designs still reuse information within a single branch. As a result, they improve optimization without
87 directly testing whether alternative information-flow topologies preserve more useful historical signals.

88 **Cross-branch interaction and graph-level feedback**

89 Cross-layer reuse in graph learning has been explored through skip connections, dense aggregation,
90 APPNP-style propagation, and Jumping Knowledge readout [17, 11, 5]. These approaches preserve
91 information from multiple depths, yet they do not directly isolate whether exchanging historical states
92 across branches or re-injecting graph summaries across block boundaries is preferable to ordinary
93 same-path residual reuse. CR-GNN is designed around exactly that distinction. It treats node-level
94 cross exchange and graph-level cross-summary propagation as explicit alternatives to standard residual
95 carryover, so that the effect of information-flow topology can be compared under one benchmark protocol.

Residual routing as a mechanism question

The present paper studies not only whether residual information is reused, but also how it is routed. A dense learnable residual gate is one possible design, but filtered routing may behave differently when the target architecture is cross-oriented or when the residual path is already strong. This motivates the residual-routing analysis in the later sections, where learnable, top-k, and sparse routing modes are evaluated under the same experimental protocol. In this sense, the paper extends a pure architecture comparison into a mechanism study of residual injection strength and selectivity.

Study positioning

The paper is best understood as a biomolecular graph-classification methods study with a mechanism-oriented empirical focus. It does not claim universal superiority of cross-residual interaction, and it does not attempt direct biological discovery. Instead, it asks a narrower and more defensible question: when does cross-residual information flow help beyond plain stacking, when does ordinary residual reuse remain the better default, and how does residual routing strategy change that answer? That positioning is important for the interpretation of the results: the goal is to characterize the boundary of usefulness for cross-residual and routed-residual designs, not to force a single winner narrative.

MATERIALS AND METHODS

This section defines the common graph-classification setting, the CR-GNN architecture family, the benchmark package, and the shared experimental protocol. The goal is to isolate where information is reused while keeping the remaining pipeline as consistent as possible.

Problem definition and notation

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ denote an undirected or directed graph, where $\mathcal{V} = \{v_1, v_2, \dots, v_n\}$ is the set of n nodes and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of edges. Each node $v_i \in \mathcal{V}$ is associated with a feature vector $\mathbf{x}_i \in \mathbb{R}^{d_{\text{in}}}$, and the node features for the entire graph are collected in $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$. The graph structure is represented using an adjacency matrix $\mathbf{A}$.

Given a training dataset $\mathcal{D} = \{(\mathcal{G}_i, y_i)\}_{i=1}^N$, the objective is to learn a mapping function $f_\theta : \mathcal{G} \rightarrow \hat{y}$ that predicts the label of an unseen graph. The present study compares five architectures under the same graph-classification pipeline:

- **PlainGNN**: sequential message passing without explicit state reuse
- **NodeResGNN**: single-branch node-level residual reuse
- **NodeCrossGNN**: two-branch node-level cross exchange
- **GraphResGNN**: graph-level residual propagation across sequential blocks
- **GraphCrossGNN**: graph-level cross-summary exchange across paired blocks

The comparison is designed to isolate where information is reused: inside one branch, across parallel branches, or through pooled graph summaries passed between blocks.

Shared message passing and graph-level learning

Let Φ denote a message-passing operator. For a chosen backbone, node representations are updated over L layers as

$$\mathbf{h}_i^{(\ell+1)} = \sigma \left(\mathbf{W}^{(\ell)} \cdot \text{AGGREGATE}_\Phi \left(\left\{ \mathbf{h}_j^{(\ell)} : j \in \mathcal{N}(i) \right\}, \mathbf{h}_i^{(\ell)} \right) + \mathbf{b}^{(\ell)} \right). \quad (1)$$

After L layers, a readout function R aggregates node-level representations into a graph embedding

$$\mathbf{h}_\mathcal{G} = R \left(\left\{ \mathbf{h}_i^{(L)} : i = 1, \dots, n \right\} \right). \quad (2)$$

The classifier produces

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}_{\text{clf}} \mathbf{h}_\mathcal{G} + \mathbf{b}_{\text{clf}}), \quad (3)$$

and the model is trained end-to-end by minimizing cross-entropy:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c}). \quad (4)$$

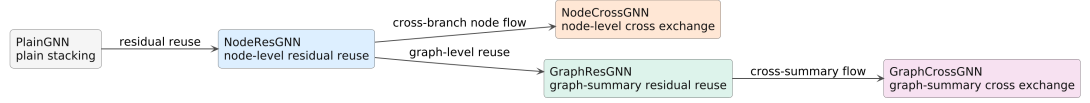


Figure 1. Main model comparison in this paper. The five architectures differ only in how intermediate node states or graph summaries are reused, which allows a controlled study of residual and cross-residual information flow.

CR-GNN architecture family

CR-GNN is defined as a compact family of graph classification architectures. All variants share the same input projection, stacked message-passing layers, graph-level pooling, and linear classifier; they differ only in how intermediate node states and graph-level summaries are reused across depth and across branches.

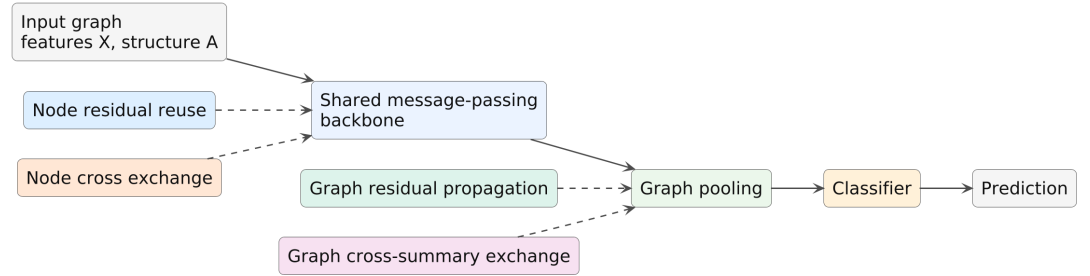


Figure 2. Schematic view of the CR-GNN pipeline. All models share the same graph-classification backbone, while node-level reuse and graph-level summary reuse define the residual and cross-residual variants.

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a graph with node feature matrix $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$ and adjacency structure $\mathbf{A}$. The model can be instantiated with one of several standard message-passing operators,

$$\Phi \in \{\text{GCN}, \text{GAT}, \text{Graph-Transformer}\},$$

and reuses the same operator type throughout a model instance. For a hidden width d , every branch starts with an input projection

$$\mathbf{H}^{(0)} = \text{ReLU}(\Phi_{\text{in}}(\mathbf{X}, \mathbf{A})), \quad (5)$$

followed by L hidden layers and a readout

$$\mathbf{g} = \text{Pool}(\mathbf{H}^{(L)}). \quad (6)$$

Single-branch and node-level residual variants

PlainGNN. The plain baseline stacks L copies of the same operator without residual reuse:

$$\mathbf{H}^{(\ell+1)} = \psi\left(\text{ReLU}\left(\Phi_{\ell}\left(\mathbf{H}^{(\ell)}, \mathbf{A}\right)\right)\right). \quad (7)$$

NodeResGNN. The first residual variant stores the previous hidden state within the same branch:

$$\mathbf{H}^{(\ell+1)} = \psi\left(\text{ReLU}\left(\Phi_{\ell}\left(\mathbf{H}^{(\ell)} + \mathbf{H}^{(\ell-1)}, \mathbf{A}\right)\right)\right). \quad (8)$$

NodeCrossGNN. The node-level cross-residual model maintains two parallel branches with separate input projections:

$$\mathbf{H}_1^{(\ell+1)} = \psi\left(\text{ReLU}\left(\Phi_{\ell}^{(1)}\left(\mathbf{H}_1^{(\ell)} + \tilde{\mathbf{H}}_2^{(\ell)}, \mathbf{A}\right)\right)\right), \quad (9)$$

$$\mathbf{H}_2^{(\ell+1)} = \psi\left(\text{ReLU}\left(\Phi_{\ell}^{(2)}\left(\mathbf{H}_2^{(\ell)} + \tilde{\mathbf{H}}_1^{(\ell)}, \mathbf{A}\right)\right)\right). \quad (10)$$

The final node representation is the branch sum

$$\mathbf{H}^{(L)} = \mathbf{H}_1^{(L)} + \mathbf{H}_2^{(L)}. \quad (11)$$

152 **Graph-level residual propagation**

153 **GraphResGNN.** This variant chains three graph-conditioned blocks:

$$\mathbf{g}^{(t+1)} = B_{t+1}(\mathbf{X}, \mathbf{A}, \mathbf{g}^{(t)}), \quad t \in \{1, 2\}. \quad (12)$$

154 **GraphCrossGNN.** Four blocks are organized as two sequential pairs. Within each stage, the two
155 block outputs are swapped and reused as the next stage’s graph-level input:

$$(\mathbf{g}_1^{(t)}, \mathbf{g}_2^{(t)}) = (B_{2t-1}(\mathbf{X}, \mathbf{A}, \mathbf{g}_1^{(t-1)}), B_{2t}(\mathbf{X}, \mathbf{A}, \mathbf{g}_2^{(t-1)})), \quad (13)$$

$$(\mathbf{g}_1^{(t+1)}, \mathbf{g}_2^{(t+1)}) = (\mathbf{g}_2^{(t)}, \mathbf{g}_1^{(t)}). \quad (14)$$

156 The final graph embedding is

$$\mathbf{g}_{\text{final}} = \mathbf{g}_1^{(T)} + \mathbf{g}_2^{(T)}. \quad (15)$$

157 Residual routing mechanisms

158 Beyond the architecture family itself, this study evaluates how residual information is injected. Three
159 routing modes are used in the targeted ablations:

- 160 • **Learnable routing:** dense residual injection modulated by an adaptive gate.
- 161 • **Top-k routing:** only the strongest residual channels are retained before injection.
- 162 • **Sparse routing:** weak residual coefficients are suppressed, leaving a filtered residual path.

163 These routing modes are not used to redefine the core architecture family. Instead, they form a mechanism
164 layer on top of selected residual and cross-residual targets in the experimental design.

165 Datasets

166 To test the proposed architecture in a biologically meaningful setting, the benchmarks are organized into a
167 unified two-layer package.

168 The main biological benchmark consists of PROTEINS, DD, and ENZYMES [4, 13]. The supple-
169 mentary robustness package consists of MUTAG, AIDS, and Mutagenicity. This organization keeps the
170 main biological claim centered on protein-oriented datasets while still testing structural robustness on
171 additional molecular graphs.

172 **PROTEINS** PROTEINS contains graph representations of proteins, where nodes denote secondary
173 structure elements and edges encode neighborhood relations between them. The task is binary graph
174 classification. The dataset contains 1,113 graphs, 2 classes, and 3 input features per node.

175 **DD** DD is a protein structure dataset in which each graph corresponds to a protein and nodes represent
176 amino acids or structure-related units linked by proximity-derived edges. The task is binary classification.
177 DD contains 1,178 graphs, 2 classes, and 89 input features.

178 **ENZYMES** ENZYMES is a 6-class enzyme classification benchmark with 600 graphs and 3 input
179 features. It preserves an enzyme-oriented biological interpretation while remaining directly comparable
180 to the other graph classification benchmarks in the study.

181 All active datasets are used through a unified benchmark interface [4, 13]. We keep preprocessing
182 intentionally light and do not introduce task-specific feature engineering, graph augmentation, or edge
183 redesign in the final protocol.

184 All reported results follow the same stratified two-stage evaluation design:

- 185 • **Outer evaluation:** stratified 5-fold cross-validation at the graph level
- 186 • **Inner validation:** a stratified validation subset sampled from the training fold with validation ratio
187 0.1

Table 1. Unified summary of the main biological benchmark package.

Dataset	Source	Task	Split	#Graphs	#Cls.	Feat.	Avg. N / E	Role
PROTEINS	TU	classification	5-fold CV + val	1113	2	3	39.1 / 72.8	main
DD	TU	classification	5-fold CV + val	1178	2	89	284.3 / 715.7	main
ENZYMES	TU	classification	5-fold CV + val	600	6	3	32.6 / 62.1	main

Table 2. Supplementary robustness datasets retained outside the core biological narrative.

Dataset	Source	Task	Split	#Graphs	#Cls.	Feat.	Avg. N / E	Role
MUTAG	TU	classification	5-fold CV + val	188	2	7	17.9 / 19.8	supp.
AIDS	TU	classification	5-fold CV + val	2000	2	38	15.7 / 16.2	supp.
Mutagenicity	TU	classification	5-fold CV + val	4337	2	14	30.0 / 30.6	supp.

The primary evaluation metric is graph classification accuracy on the held-out test fold. For fold k with test set $\mathcal{T}_k$, the fold accuracy is

$$\text{Acc}_k = \frac{1}{|\mathcal{T}_k|} \sum_{(\mathcal{G}_i, y_i) \in \mathcal{T}_k} \mathbf{1}[\hat{y}_i = y_i]. \quad (16)$$

Across the five folds, we report the mean accuracy

$$\overline{\text{Acc}} = \frac{1}{5} \sum_{k=1}^5 \text{Acc}_k \quad (17)$$

and the corresponding standard deviation

$$\sigma_{\text{Acc}} = \sqrt{\frac{1}{5} \sum_{k=1}^5 (\text{Acc}_k - \overline{\text{Acc}})^2}. \quad (18)$$

Compared methods and backbone operators

The full paper-facing benchmark compares nine methods. The CR-GNN family contains five models: PlainGNN, NodeResGNN, NodeCrossGNN, GraphResGNN, and GraphCrossGNN. Four external baselines are also included in the benchmark package: GraphSAGEBaseline, GINBaseline, JKNetBaseline, and APPNPBaseline.

Each method is evaluated under four message-passing operators:

- **GCNConv**
- **GATConv**
- **GINConv**
- **SAGEConv**

Training configuration and implementation details

All reported runs follow the same versioned experimental workflow, with V3 as the active paper-facing source. The recommended workflow entry is `py/run_final_v3_pipeline.py`, and the associated outputs are stored in `logs/V3`, `records/V3`, and `runs/V3`. Preprocessing is intentionally light, and the final protocol does not add task-specific feature engineering, graph augmentation, or custom edge redesign.

Evaluation metrics and statistical reporting

The primary metric is graph classification accuracy on the held-out test fold. Main benchmark and targeted ablation results are reported as five-fold mean accuracy with standard deviation. In the final paper, the five-fold V3 experiments are treated as the main confirmatory evidence, while older single-fold parameter scans are used only as supporting exploratory analyses.

213 **EXPERIMENTAL DESIGN**

214 This section separates the organization of the empirical study from the general model and implementation
215 details in Materials and Methods. The goal is to clarify how the final benchmark and targeted ablations
216 are structured, which results are confirmatory five-fold evidence, and which analyses are exploratory only.

217 **Overall benchmark design**

218 The active paper-facing experiment source is version V3. The final benchmark evaluates six datasets, nine
219 methods, four message-passing operators, and five outer folds under one unified protocol. This yields the
220 main comparison matrix used throughout the paper. For readability, the main text reports compressed
221 summary tables and figures, while the complete matrix is moved to the appendix.

222 **Biological core and supplementary benchmark split**

223 The benchmark is organized into two layers. The biological core consists of PROTEINS, DD, and
224 ENZYMES, which anchor the main biomolecular claim of the paper. A supplementary robustness
225 package extends the analysis to MUTAG, AIDS, and Mutagenicity. This split avoids mixing the strongest
226 biological narrative with the broader molecular robustness checks and makes the final interpretation more
227 disciplined.

228 **Gate ablation design**

229 The first targeted ablation studies adaptive gate control on the strongest graph-level residual line. The target
230 configuration is AIDS + GraphResGNN + GINConv, and the compared settings are a learnable gate
231 and three fixed coefficients (0.0, 0.5, and 1.0). This experiment is designed to test whether adaptive
232 residual injection is stronger or more stable than fixed-strength graph-level reuse.

233 **Cross-gate ablation design**

234 The second ablation focuses on settings where a cross architecture is already locally favored. Six
235 dataset + model + operator targets are evaluated under the same four gate settings: AIDS +
236 NodeCrossGNN + GATConv, DD + NodeCrossGNN + GATConv, DD + NodeCrossGNN +
237 GINConv, ENZYMES + NodeCrossGNN + GATConv, MUTAG + GraphCrossGNN + GATConv,
238 and Mutagenicity + NodeCrossGNN + GATConv. The purpose is not to prove that learnable
239 gating always wins, but to test whether adaptive gate control remains robust when cross exchange is
240 already competitive.

241 **Residual-routing ablation design**

242 The main mechanism study targets representative residual- and cross-oriented lines: AIDS + GraphResGNN
243 + GINConv, PROTEINS + GraphResGNN + SAGEConv, AIDS + NodeCrossGNN + GATConv,
244 and DD + NodeCrossGNN + GATConv. Five routing settings are compared under full five-fold
245 evaluation: learnable, topk_0p25, topk_0p5, sparse_0p02, and sparse_0p05. These exper-
246 iments are designed to answer whether mild filtering of residual injection can outperform dense learnable
247 routing on selected targets.

248 **Residual-parameter sweep design**

249 To extend the routing analysis, two additional settings are evaluated on the same four targets: topk_0p75
250 and sparse_0p1. This sweep is used to test whether the optimal top-k ratio is target-dependent and
251 whether more aggressive sparsity harms otherwise strong residual lines.

252 **Exploratory sensitivity analysis**

253 Older single-fold sensitivity scans are retained as supporting analyses rather than as main evidence. These
254 scans focus on cross models on PROTEINS, DD, and ENZYMES and vary depth, dropout, and learning
255 rate on fold 0 only. In the final paper, they should be interpreted as optimization-trend evidence and
256 clearly separated from the five-fold confirmatory experiments in V3.

257 **RESULTS**

258 This section reports the final empirical evidence in three layers: the full benchmark, the targeted gate and
259 residual-routing studies, and the exploratory sensitivity analyses. The main benchmark and the targeted

260 ablations use the paper-facing V3 five-fold results, whereas the older sensitivity scans are retained only as
 261 supporting trend analyses.

262 Overall benchmark performance

263 The final benchmark evaluates six datasets, nine methods, four operators, and five outer folds under
 264 one unified protocol. For readability, the main text reports compressed summary tables over the active
 265 biological and supplementary datasets, while the complete matrix is reproduced in the appendix.

Table 3. Main biological benchmark results. Numbers are mean best test accuracy $\pm$ standard deviation.

Model	PROTEINS	DD	ENZYMES
PlainGNN	0.7053 $\pm$ 0.0271	0.7165 $\pm$ 0.0179	0.2450 $\pm$ 0.0407
NodeResGNN	0.6945 $\pm$ 0.0509	0.7207 $\pm$ 0.0259	0.2650 $\pm$ 0.0661
NodeCrossGNN	0.6963 $\pm$ 0.0433	0.7198 $\pm$ 0.0178	0.3000 $\pm$ 0.0580
GraphResGNN	0.7223 $\pm$ 0.0367	0.6934 $\pm$ 0.0458	0.3117 $\pm$ 0.0746
GraphCrossGNN	0.6864 $\pm$ 0.0465	0.7097 $\pm$ 0.0309	0.3017 $\pm$ 0.0750

266 Table 3 anchors the paper’s biological argument. GraphResGNN is best on PROTEINS and ENZYMES, whereas
 267 NodeResGNN is best on DD, so the core package already favors residual reuse over plain stacking. The cross-residual
 268 variants remain competitive, but on the main biological datasets they mostly narrow the gap instead of overturning the
 269 residual baseline.

Table 4. Supplementary robustness benchmark results.

Model	MUTAG	AIDS	Mutagenicity
PlainGNN	0.7236 $\pm$ 0.0486	0.8455 $\pm$ 0.0257	0.7814 $\pm$ 0.0235
NodeResGNN	0.7343 $\pm$ 0.0734	0.8690 $\pm$ 0.0354	0.7897 $\pm$ 0.0139
NodeCrossGNN	0.7397 $\pm$ 0.0676	0.8735 $\pm$ 0.0412	0.8033 $\pm$ 0.0159
GraphResGNN	0.7290 $\pm$ 0.0585	0.9160 $\pm$ 0.0121	0.8022 $\pm$ 0.0220
GraphCrossGNN	0.7129 $\pm$ 0.0539	0.8710 $\pm$ 0.0412	0.8029 $\pm$ 0.0182

270 Table 4 broadens the scope beyond the protein-oriented core. NodeCrossGNN becomes strongest on MUTAG and
 271 Mutagenicity, while GraphResGNN remains best on AIDS. This mixed pattern shows why cross-residual routing is
 272 worth keeping in the comparison, but also why the paper avoids claiming that cross routing is universally stronger
 273 than residual reuse.

Table 5. Winner summary, optimization depth, and parameter count for each dataset.

Dataset	Winner	Mean Acc	Mean Best Epoch	Params
PROTEINS	GraphResGNN	0.7223	58.8	51208
DD	NodeResGNN	0.7207	25.4	22530
ENZYMES	GraphResGNN	0.3117	91.4	52248
MUTAG	NodeCrossGNN	0.7397	88.2	34434
AIDS	GraphResGNN	0.9160	83.8	57928
Mutagenicity	NodeCrossGNN	0.8033	131.2	35330

274 Table 5 turns the six benchmark outcomes into a compact cross-dataset summary. The winners change with dataset
 275 regime, the best epoch ranges from 25.4 to 131.2, and the largest parameter count does not always produce the best
 276 mean accuracy. This supports the paper’s main methodological point that information-flow design matters more than
 277 simply selecting the heaviest model.

278 Taken together, the three tables show a consistent hierarchy. Residual reuse is still the stronger default
 279 family in the protein-oriented core, cross-residual routing becomes selectively useful on supplementary
 280 molecular datasets, and the winner summary confirms that no single complexity trend explains the results
 281 by itself.

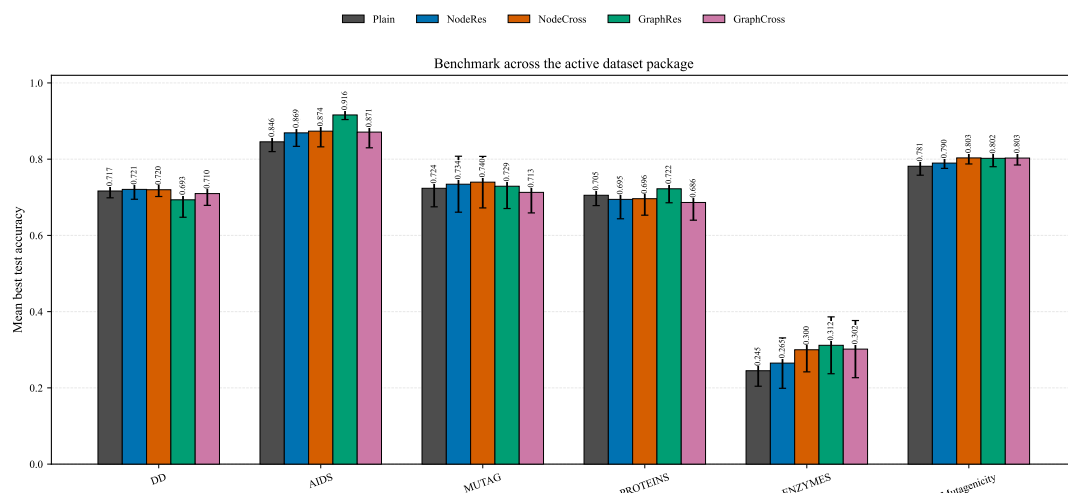


Figure 3. Benchmark over the active dataset package. Each bar reports mean best test accuracy, the numeric label above each bar gives the exact mean value, and error bars show the fold-level standard deviation.

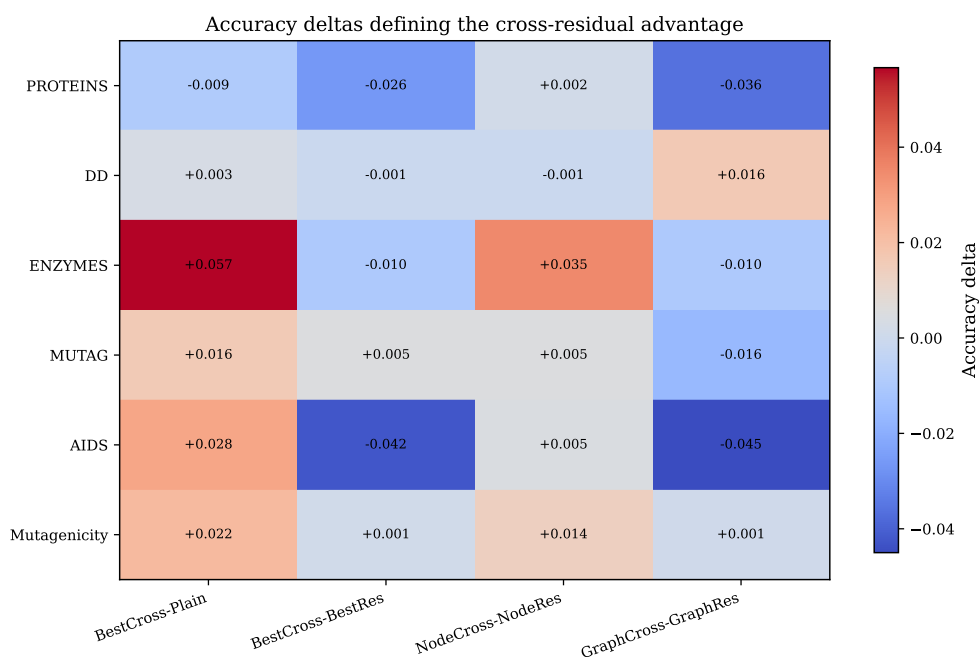


Figure 4. Accuracy deltas that define the practical value of cross-residual routing. Positive values indicate an advantage for the cross design, while negative values indicate that the corresponding plain or residual alternative remains stronger.

Figure 3 and Figure 4 make the same point from complementary angles: cross-residual routing is usually better than plain stacking, but it is not consistently stronger than the best residual alternative.

Results on the biological core datasets

Residual reuse remains the strongest default family in the reported suite. Residual variants win PROTEINS, DD, ENZYMES, and AIDS. In particular, GraphResGNN is best on PROTEINS (0.7223 ± 0.0367), ENZYMES (0.3117 ± 0.0746), and AIDS (0.9160 ± 0.0121), while NodeResGNN is best on DD (0.7207 ± 0.0259).

Cross-residual design is selectively strong, not universally dominant. Cross variants still win completed datasets such as MUTAG and Mutagenicity. More importantly, the best cross variant beats PlainGNN on most completed datasets. Therefore, cross routing is clearly more than a design curiosity, but the evidence does not support the stronger claim that cross universally outperforms residual reuse.

Node-level cross is more reliable than graph-level cross. Across the completed datasets, NodeCrossGNN outperforms NodeResGNN more often than GraphCrossGNN outperforms GraphResGNN. This suggests that exchanging intermediate node-level states is the more robust cross-residual design in the present study, while graph-level cross routing is more dataset-sensitive.

Plain stacking is no longer a credible final baseline. PlainGNN remains competitive on PROTEINS and DD, but it never wins a dataset in the final suite and is usually surpassed by either a residual or a cross-residual variant.

Topic-focused results for PROTEINS, DD, and ENZYMES

The topic-facing interpretation is anchored in the protein-oriented core datasets.

On **PROTEINS**, GraphResGNN is clearly strongest, indicating that repeated graph-level residual refinement is the most effective way to preserve useful global structure. The best cross model here is NodeCrossGNN at 0.6963 ± 0.0433 , which improves slightly over NodeResGNN but remains below both PlainGNN and GraphResGNN.

On **DD**, NodeResGNN and NodeCrossGNN are nearly tied, with NodeResGNN retaining a narrow lead. This indicates that node-level hidden-state reuse is important on large sparse protein graphs, but cross exchange does not produce a decisive improvement over simpler residual reuse.

On **ENZYMES**, cross variants become more attractive. GraphCrossGNN reaches 0.3017 ± 0.0750 and NodeCrossGNN reaches 0.3000 ± 0.0580 , both clearly above PlainGNN at 0.2450 ± 0.0407 . Even so, GraphResGNN still remains the best model.

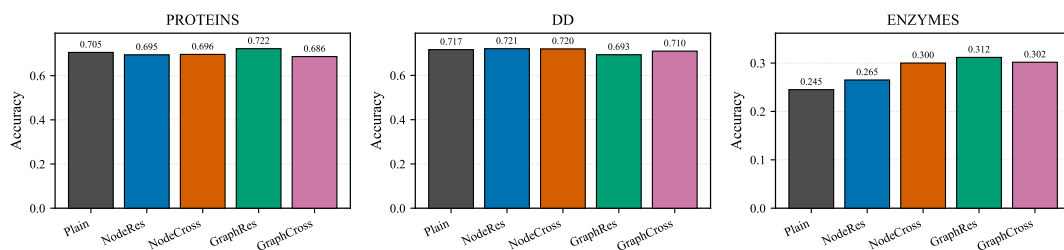


Figure 5. Focused comparison on the completed topic-facing TU datasets. The pattern is mixed rather than one-sided: graph-level residual refinement is strongest on PROTEINS and ENZYMES, while DD favors node-level reuse, with NodeCrossGNN remaining competitive but not dominant.

Results on the supplementary robustness datasets

The supplementary package broadens the scope beyond the biological core without changing the main conclusion. NodeCrossGNN is strongest on MUTAG and Mutagenicity, whereas GraphResGNN remains strongest on AIDS. This pattern preserves the paper’s main boundary claim: cross-residual interaction has selective practical value, but the residual family remains the more reliable default reference across the full suite.

Gate ablation results

The gate ablations are designed to test whether adaptive control of residual injection is useful once the main benchmark has identified strong target configurations. On the strongest graph-level residual line, AIDS + GraphResGNN + GINConv, the comparison among learnable and fixed gates is the cleanest direct test of adaptive graph-level reuse. Table 6 shows that the learnable gate is the strongest setting, although the fixed coefficient 0.5 remains very close. The cross-gate ablation extends the same comparison to six settings where a cross architecture is already locally favored.

The intended interpretation is constrained. The gate study does not need to show that learnable gating is best on every target. Instead, the relevant question is whether adaptive control is generally more robust than fixed injection strengths across the selected residual and cross-residual settings. Table 7 shows a

Table 6. AIDS gate ablation on GraphResGNN + GINConv. Numbers are five-fold mean accuracy $\pm$ standard deviation.

Gate setting	Mean accuracy
Learnable	0.9620 $\pm$ 0.0089
Fixed 0.0	0.9150 $\pm$ 0.0138
Fixed 0.5	0.9605 $\pm$ 0.0144
Fixed 1.0	0.9480 $\pm$ 0.0300

Table 7. Cross-gate ablation on six cross-favored targets. Numbers are five-fold mean accuracy $\pm$ standard deviation.

Dataset	Target model/operator	Learnable	Fixed 0.0	Fixed 0.5	Fixed 1.0
AIDS	NodeCrossGNN + GATConv	0.9045 $\pm$ 0.0165	0.9075 $\pm$ 0.0065	0.9195 $\pm$ 0.0114	0.9105 $\pm$ 0.0241
DD	NodeCrossGNN + GATConv	0.7173 $\pm$ 0.0271	0.7053 $\pm$ 0.0361	0.7156 $\pm$ 0.0225	0.7147 $\pm$ 0.0288
DD	NodeCrossGNN + GINConv	0.7071 $\pm$ 0.0306	0.7012 $\pm$ 0.0198	0.7232 $\pm$ 0.0207	0.7113 $\pm$ 0.0277
ENZYMES	NodeCrossGNN + GATConv	0.2800 $\pm$ 0.0629	0.2850 $\pm$ 0.0627	0.2550 $\pm$ 0.0674	0.2650 $\pm$ 0.0690
MUTAG	GraphCrossGNN + GATConv	0.7558 $\pm$ 0.0495	0.7451 $\pm$ 0.0407	0.7450 $\pm$ 0.0416	0.7504 $\pm$ 0.0779
Mutagenicity	NodeCrossGNN + GATConv	0.8114 $\pm$ 0.0110	0.8033 $\pm$ 0.0192	0.8010 $\pm$ 0.0130	0.8036 $\pm$ 0.0078

mixed result: learnable gating is best on three of the six cross-favored targets, while fixed gates remain competitive and can even be superior on the other three targets. This supports a robustness claim, not a universal-optimum claim.

Residual-routing results

The residual-routing analysis is the main mechanism extension of the paper. Four representative targets compare learnable, top-k, and sparse routing modes under the same five-fold protocol. The clearest overall signal is that there is no universal winner. Table 8 shows that learnable remains best on the strongest graph-level residual line (AIDS + GraphResGNN + GINConv), but sparse routing is strongest on three of the four representative targets, with sparse_0p05 emerging as the most consistent sparse setting.

This pattern supports a more precise conclusion than the architecture-only benchmark alone. The important question is not only whether historical information is reused, but also how aggressively that information is filtered before reuse. Table 9 compresses the same result by routing family: the sparse family is strongest on three of the four representative targets, while the learnable family remains strongest on the best graph-level residual line.

Residual-parameter sweep results

The additional routing settings topk_0p75 and sparse_0p1 refine the previous result rather than changing it. Table 8 shows that the strongest top-k ratio varies by target, which indicates that top-k routing is not governed by one globally optimal retention level. The sparse sweep also shows a clear failure mode: aggressive sparsity can damage otherwise strong residual lines, especially on the strongest graph-level residual target, where sparse_0p1 drops to 0.8315 $\pm$ 0.0549.

Exploratory sensitivity analysis

Auxiliary sensitivity checks over depth, dropout, and learning rate on PROTEINS, DD, and ENZYMES do not materially change the main conclusion. On PROTEINS and DD, moderate tuning changes affect the gap between the two cross variants, but they do not overturn the broader conclusion that residual baselines remain the stronger default reference. On ENZYMES, the validation signal is visibly noisier, and promising local settings did not reproduce the original full-suite cross results under the final 5-fold average.

DISCUSSION

The main empirical message is now more precise than in the earlier draft. Cross-residual routing is usually beneficial relative to plain stacking, but the gain over residual baselines is conditional and

Table 8. Residual-routing and parameter-sweep summary on four representative targets. Numbers are five-fold mean accuracy $\pm$ standard deviation.

Target	Learnable	Top-k 0.25	Top-k 0.5	Top-k 0.75	Sparse 0.02	Sparse 0.05	Sparse 0.1	Best
AIDS + GraphResGNN + GINConv	0.9500 $\pm$ 0.0087	0.9075 $\pm$ 0.0237	0.9330 $\pm$ 0.0251	0.9240 $\pm$ 0.0287	0.9470 $\pm$ 0.0181	0.9250 $\pm$ 0.0152	0.8315 $\pm$ 0.0549	Learnable
PROTEINS + GraphResGNN + SAGEConv	0.7178 $\pm$ 0.0411	0.7241 $\pm$ 0.0407	0.7151 $\pm$ 0.0397	0.7214 $\pm$ 0.0318	0.7160 $\pm$ 0.0248	0.7259 $\pm$ 0.0475	0.7088 $\pm$ 0.0521	Sparse 0.05
AIDS + NodeCrossGNN + GATConv	0.8970 $\pm$ 0.0491	0.9160 $\pm$ 0.0128	0.8960 $\pm$ 0.0483	0.8960 $\pm$ 0.0489	0.9060 $\pm$ 0.0268	0.9180 $\pm$ 0.0162	0.9025 $\pm$ 0.0139	Sparse 0.05
DD + NodeCrossGNN + GATConv	0.7080 $\pm$ 0.0172	0.7062 $\pm$ 0.0249	0.7029 $\pm$ 0.0160	0.7164 $\pm$ 0.0248	0.7147 $\pm$ 0.0259	0.7181 $\pm$ 0.0196	0.7105 $\pm$ 0.0243	Sparse 0.05

Table 9. Family-wise routing winners.

Target	Best learnable	Best top-k	Best sparse	Overall winner
AIDS + GraphResGNN + GINConv	0.9500	0.9330	0.9470	Learnable
PROTEINS + GraphResGNN + SAGEConv	0.7178	0.7241	0.7259	Sparse
AIDS + NodeCrossGNN + GATConv	0.8970	0.9160	0.9180	Sparse
DD + NodeCrossGNN + GATConv	0.7080	0.7164	0.7181	Sparse

dataset-dependent. The present evidence therefore supports CR-GNN as a meaningful biomolecular graph-classification design space rather than as a universally dominant model family.

Cross-residual gains are conditional

The benchmark results do not support the strongest possible claim that cross-residual interaction should replace residual reuse as the default choice. Instead, they support a narrower statement: cross-residual design is often better than plain stacking and can be distinctly useful on selected datasets, but ordinary residual reuse remains the stronger default family across the full biomolecular package.

Node-level and graph-level reuse behave differently

The distinction between node-level and graph-level information reuse is also practically important. Node-level cross exchange is the more stable cross-oriented design, whereas graph-level cross routing is more sensitive to dataset regime and does not consistently match the strongest graph-level residual lines. This suggests that the location of reuse is at least as important as the existence of an additional reuse path.

Residual routing is part of the mechanism story

The residual-routing ablations add an important qualification to the main architecture comparison. The question is not only whether residual information should be reused, but also whether it should be reused densely, sparsified, or filtered by top-k selection. The current results show that routing strategy is target-dependent: learnable dense routing remains strongest on the best graph-level residual line, while mild sparse routing is frequently advantageous on cross-oriented settings. This turns the paper into a mechanism study rather than a pure architecture ranking exercise.

Evidence hierarchy and study limitations

The strongest evidence in the paper comes from the five-fold V3 benchmark and the five-fold targeted ablations. By contrast, the older single-fold sensitivity scans are useful only for optimization interpretation. They should not be treated as the same class of quantitative evidence. A second limitation is domain scope: although the datasets are biomolecular or molecular and the biological core is meaningful, the study remains benchmark-centered and structure-centric. It does not yet justify broader claims about direct biological discovery, omics integration, or task-specific biological interpretation.

Future directions

The next research step is to test the same information-flow ideas with richer biological inputs, such as sequence-derived embeddings, ontology annotations, or expression-related signals. In parallel, the residual-routing perspective introduced here could be extended to broader biomolecular graph tasks where graph topology must be combined with stronger domain supervision rather than benchmark-only evaluation.

CONCLUSIONS

This paper studies CR-GNN as a biomolecular graph-classification framework rather than as a claim of universal model superiority. The final evidence supports a narrower but stronger conclusion: cross-residual interaction is useful, but residual variants remain the strongest default family in the reported benchmark suite.

The main contribution is therefore not a universally dominant architecture, but a controlled comparison of information-flow mechanisms and residual-routing strategies. Across the biological core, GraphResGNN is strongest on PROTEINS and ENZYMES, whereas NodeResGNN is strongest on DD. Cross-residual variants still provide useful gains over plain stacking on selected datasets, but they do not replace residual baselines as the main point of reference.

The routing analysis sharpens that result further. Learnable dense routing remains strongest on the best graph-level residual line, while mild sparse routing is frequently effective on cross-oriented settings. Taken together, these results position CR-GNN as a practical biomolecular graph-classification design space and provide a methodological basis for future work on richer biological graph-learning settings.

ACKNOWLEDGMENTS

The authors thank the open-source graph learning community and the maintainers of the benchmark resources used in this study. They also thank the reviewers and editors for their time and feedback.

A WHY RESIDUAL REUSE CAN SLOW OVER-SMOOTHING

This appendix provides a simplified derivation that explains why residual reuse can mitigate over-smoothing. The goal is not to claim a full theorem for the nonlinear models in Section , but to show that the proposed residual pathways change the effective propagation filter from a single high-order power to a lower-order polynomial mixture.

A.1 Linearized Plain Propagation

Consider the standard linearized message-passing update

$$\mathbf{H}^{(\ell+1)} = \mathbf{S}\mathbf{H}^{(\ell)}, \quad (19)$$

where $\mathbf{S}$ is a normalized graph propagation operator and $\mathbf{H}^{(0)} = \mathbf{X}$. Ignoring nonlinearities and trainable weights yields

$$\mathbf{H}^{(\ell)} = \mathbf{S}^\ell \mathbf{X}. \quad (20)$$

If $\mathbf{S}$ is diagonalizable as $\mathbf{S} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^{-1}$, then

$$\mathbf{H}^{(\ell)} = \mathbf{U}\mathbf{\Lambda}^\ell\mathbf{U}^{-1}\mathbf{X}. \quad (21)$$

For normalized propagation, the dominant eigenvalue is typically close to 1, while the remaining eigenvalues satisfy $|\lambda_i| < 1$. As ℓ grows, the factors λ_i^ℓ suppress non-dominant components, so node embeddings gradually collapse toward a low-dimensional smooth subspace. This is the standard over-smoothing effect: nodes become increasingly difficult to distinguish because repeated propagation attenuates the higher-frequency components that encode local differences.

A.2 Node-Level Residual Reuse

Now consider a residual reuse update

$$\mathbf{H}^{(\ell+1)} = \mathbf{S}(\mathbf{H}^{(\ell)} + \beta\mathbf{H}^{(\ell-1)}), \quad (22)$$

where $\beta \geq 0$ controls the strength of the reused hidden state. In this study, we set $\beta = 1$.

Define $\mathbf{H}^{(\ell)} = P_\ell(\mathbf{S})\mathbf{X}$ with $P_0(\mathbf{S}) = \mathbf{I}$ and $P_1(\mathbf{S}) = \mathbf{S}$. Then Eq. (22) implies the polynomial recurrence

$$P_{\ell+1}(\mathbf{S}) = \mathbf{S}(P_\ell(\mathbf{S}) + \beta P_{\ell-1}(\mathbf{S})). \quad (23)$$

428 The first few terms are

$$P_1(\mathbf{S}) = \mathbf{S}, \quad (24)$$

$$P_2(\mathbf{S}) = \mathbf{S}^2 + \beta \mathbf{S}, \quad (25)$$

$$P_3(\mathbf{S}) = \mathbf{S}^3 + 2\beta \mathbf{S}^2, \quad (26)$$

$$P_4(\mathbf{S}) = \mathbf{S}^4 + 3\beta \mathbf{S}^3 + \beta^2 \mathbf{S}^2. \quad (27)$$

429 Therefore, residual reuse no longer applies only the single power $\mathbf{S}^\ell$; instead, it produces a mixture of
 430 lower-order and higher-order propagation terms. In spectral coordinates, each eigen-component evolves
 431 as

$$z_i^{(\ell+1)} = \lambda_i(z_i^{(\ell)} + \beta z_i^{(\ell-1)}), \quad (28)$$

432 which is fundamentally different from the plain recurrence $z_i^{(\ell)} = \lambda_i^\ell z_i^{(0)}$. The plain model attenuates every
 433 non-dominant mode with a pure monomial factor, whereas the residual model preserves a polynomial
 434 combination of powers of λ_i . As a result, informative mid-frequency components are not suppressed as
 435 aggressively at the same depth.

436 A.3 Implication for Over-smoothing

437 The derivation above suggests the following interpretation. Plain stacking behaves like a repeated low-pass
 438 filter whose order increases monotonically with depth. Residual reuse changes that filter into a polynomial
 439 response that still smooths the graph, but preserves more low-order propagation information from earlier
 440 layers. This does not eliminate smoothing, yet it slows the rate at which node representations collapse to
 441 an almost indistinguishable subspace. In other words, residual reuse can improve depth tolerance because
 442 it reduces the dominance of the highest propagation order.

443 A.4 Graph-Level Residual Passing

444 The same intuition extends to graph-level residual propagation. In the graph-conditioned variants, a
 445 pooled summary from the previous block is broadcast back to the current block, so the hidden state is no
 446 longer determined only by repeated applications of $\mathbf{S}$. A simplified affine form is

$$\mathbf{H}_t^{(\ell+1)} = \mathbf{S}\mathbf{H}_t^{(\ell)} + \mathbf{1}(\mathbf{g}_{t-1})^\top, \quad (29)$$

447 where $\mathbf{g}_{t-1}$ is the preceding graph summary. The additive term injects a lower-order global context at every
 448 block and prevents the representation from being governed solely by deeper powers of the propagation
 449 operator. This offers a second pathway for delaying over-smoothing, although the empirical sections
 450 show that graph-level reuse is more sensitive to dataset structure and computational cost than the lighter
 451 node-level residual design.

452 A.5 Scope of the Derivation

453 This appendix uses a linearized analysis and therefore abstracts away nonlinear activations, dropout,
 454 optimizer dynamics, and operator-specific details. It should be read as an explanatory mechanism rather
 455 than as a complete proof. Still, it aligns with the empirical observation that node-level residual reuse
 456 remains more stable than plain stacking as depth increases, and it provides a compact theoretical rationale
 457 for why reusing earlier states can alleviate over-smoothing.

458 B COMPLETE BENCHMARK TABLES

459 For completeness, we reproduce the final full-suite benchmark tables in the appendix. These are the same
 460 results discussed in Section , but collecting them here makes the full comparison easier to inspect during
 461 review.

Table 10. Main biological benchmark results. Numbers are mean best test accuracy $\pm$ standard deviation.

Model	PROTEINS	DD	ENZYMES
PlainGNN	0.7053 $\pm$ 0.0271	0.7165 $\pm$ 0.0179	0.2450 $\pm$ 0.0407
NodeResGNN	0.6945 $\pm$ 0.0509	0.7207 $\pm$ 0.0259	0.2650 $\pm$ 0.0661
NodeCrossGNN	0.6963 $\pm$ 0.0433	0.7198 $\pm$ 0.0178	0.3000 $\pm$ 0.0580
GraphResGNN	0.7223 $\pm$ 0.0367	0.6934 $\pm$ 0.0458	0.3117 $\pm$ 0.0746
GraphCrossGNN	0.6864 $\pm$ 0.0465	0.7097 $\pm$ 0.0309	0.3017 $\pm$ 0.0750

Table 11. Supplementary robustness benchmark results.

Model	MUTAG	AIDS	Mutagenicity
PlainGNN	0.7236 $\pm$ 0.0486	0.8455 $\pm$ 0.0257	0.7814 $\pm$ 0.0235
NodeResGNN	0.7343 $\pm$ 0.0734	0.8690 $\pm$ 0.0354	0.7897 $\pm$ 0.0139
NodeCrossGNN	0.7397 $\pm$ 0.0676	0.8735 $\pm$ 0.0412	0.8033 $\pm$ 0.0159
GraphResGNN	0.7290 $\pm$ 0.0585	0.9160 $\pm$ 0.0121	0.8022 $\pm$ 0.0220
GraphCrossGNN	0.7129 $\pm$ 0.0539	0.8710 $\pm$ 0.0412	0.8029 $\pm$ 0.0182

Table 12. Winner summary, optimization depth, and parameter count for each dataset.

Dataset	Winner	Mean Acc	Mean Best Epoch	Params
PROTEINS	GraphResGNN	0.7223	58.8	51208
DD	NodeResGNN	0.7207	25.4	22530
ENZYMES	GraphResGNN	0.3117	91.4	52248
MUTAG	NodeCrossGNN	0.7397	88.2	34434
AIDS	GraphResGNN	0.9160	83.8	57928
Mutagenicity	NodeCrossGNN	0.8033	131.2	35330

C TARGETED ABLATION TABLES

This section collects the confirmatory five-fold ablations that support the main mechanism claims in the paper. The tables below separate the graph-level gate study, the cross-gate study on cross-favored targets, and the residual-routing summary used in the main text.

D EXPLORATORY SENSITIVITY TABLES

This appendix section is reserved for the older single-fold sensitivity analyses over depth, dropout, and learning rate on PROTEINS, DD, and ENZYMES. These tables and figures should be explicitly labeled as exploratory, because they are useful for optimization interpretation but do not have the same evidential status as the five-fold V3 benchmark and targeted ablations.

REFERENCES

- [1] Xavier Bresson and Thomas Laurent. Residual gated graph convnets. *arXiv preprint arXiv:1711.07553*, 2017.
- [2] Tianle Cai, Jiacheng Luo, Sheng Wang, Kai Zhang, Yu Tian, Liwei Yang, Zekun Li, and Kilian Weinberger. Graphnorm: A principled approach to accelerating graph neural network training. In *International Conference on Machine Learning*, pages 1277–1287, 2021.
- [3] Yuning Chen, Xihao Wei, Pan Xu, Xinwen Wang, Ping Luo, Zhiyuan Li, and Jian Tang. Revisiting over-smoothing in deep gcns. *arXiv preprint arXiv:2003.13663*, 2020.
- [4] Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with pytorch geometric. *arXiv preprint arXiv:1903.02428*, 2019.
- [5] Hongyang Gao and Shuiwang Ji. Graph u-net. *arXiv preprint arXiv:1905.05178*, 2019.

Table 13. Fold-level AIDS gate ablation on AIDS + GraphResGNN + GINConv.

Gate setting	Fold 0	Fold 1	Fold 2	Fold 3	Fold 4	Mean $\pm$ Std
Learnable	0.9500	0.9700	0.9700	0.9675	0.9525	0.9620 $\pm$ 0.0089
Fixed 0.0	0.9225	0.9025	0.9125	0.9000	0.9375	0.9150 $\pm$ 0.0138
Fixed 0.5	0.9325	0.9625	0.9675	0.9675	0.9725	0.9605 $\pm$ 0.0144
Fixed 1.0	0.8925	0.9450	0.9700	0.9775	0.9550	0.9480 $\pm$ 0.0300

Table 14. Cross-gate ablation on six cross-favored targets. Numbers are five-fold mean accuracy $\pm$ standard deviation.

Dataset	Target model/operator	Learnable	Fixed 0.0	Fixed 0.5	Fixed 1.0
AIDS	NodeCrossGNN + GATConv	0.9045 $\pm$ 0.0165	0.9075 $\pm$ 0.0065	0.9195 $\pm$ 0.0114	0.9105 $\pm$ 0.0241
DD	NodeCrossGNN + GATConv	0.7173 $\pm$ 0.0271	0.7053 $\pm$ 0.0361	0.7156 $\pm$ 0.0225	0.7147 $\pm$ 0.0288
DD	NodeCrossGNN + GINConv	0.7071 $\pm$ 0.0306	0.7012 $\pm$ 0.0198	0.7232 $\pm$ 0.0207	0.7113 $\pm$ 0.0277
ENZYMES	NodeCrossGNN + GATConv	0.2800 $\pm$ 0.0629	0.2850 $\pm$ 0.0627	0.2550 $\pm$ 0.0674	0.2650 $\pm$ 0.0690
MUTAG	GraphCrossGNN + GATConv	0.7558 $\pm$ 0.0495	0.7451 $\pm$ 0.0407	0.7450 $\pm$ 0.0416	0.7504 $\pm$ 0.0779
Mutagenicity	NodeCrossGNN + GATConv	0.8114 $\pm$ 0.0110	0.8033 $\pm$ 0.0192	0.8010 $\pm$ 0.0130	0.8036 $\pm$ 0.0078

- [6] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. Neural message passing for quantum chemistry. In *International Conference on Machine Learning*, pages 1263–1272, 2017.
- [7] Vladimir Gligorijevic, P. Douglas Renfrew, Tomasz Kosciolk, Julia Koehler Leman, Daniel Berenberg, Tommi Vatanen, Chris Chandler, Bryn C. Taylor, Ian M. Fisk, Hera Vlamakis, Ramnik J. Xavier, Rob Knight, Kyunghyun Cho, and Richard Bonneau. Structure-based protein function prediction using graph convolutional networks. *Nature Communications*, 12(1), 2021.
- [8] William L Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in neural information processing systems*, pages 1024–1034, 2017.
- [9] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [10] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.
- [11] Johannes Klicpera, Alan Bojchevski, and Stephan Günnemann. Combining label propagation and simple models out-performs graph neural networks. *arXiv preprint arXiv:1810.05997*, 2018.
- [12] Guohao Li, Elias Muller, Abbas Thabet, and Bernard Ghanem. Deeper insights into graph convolutional networks for semi-supervised learning. *arXiv preprint arXiv:1809.02584*, 2018.
- [13] Christopher Morris, Roger Wattenhofer, and Kristian Kersting. TUDataset: A collection of benchmark datasets for learning with graphs. *arXiv preprint arXiv:2007.08663*, 2020.
- [14] Kenta Oono and Taiji Suzuki. Simple and deep graph convolutional networks. *arXiv preprint arXiv:2006.13604*, 2020.
- [15] Manon Reau, Nicolas Renaud, Li C Xue, and Alexandre M. J. J. Bonvin. DeepRank-GNN: a graph neural network framework to learn patterns in protein–protein interfaces. *Bioinformatics*, 39(1), 2022.
- [16] Yu Rong, Wenbing Huang, Tingyang Xu, and Junzhou Huang. DropEdge: Towards deep graph convolutional networks on node classification. *arXiv preprint arXiv:1907.10903*, 2019.
- [17] Keyulu Xu, Chengtao Li, Yonglong Tian, Xiao Du, Jure Leskovec, and Stefanie Jegelka. Representation learning on graphs with jumping knowledge networks. In *International Conference on Machine Learning*, pages 5449–5458, 2018.

Table 15. Residual-routing and parameter-sweep summary on four representative targets. Numbers are five-fold mean accuracy $\pm$ standard deviation.

Target	Learnable	Top-k 0.25	Top-k 0.5	Top-k 0.75	Sparse 0.02	Sparse 0.05	Sparse 0.1	Best
AIDS + GraphResGNN + GINConv	0.9500 $\pm$ 0.0087	0.9075 $\pm$ 0.0237	0.9330 $\pm$ 0.0251	0.9240 $\pm$ 0.0287	0.9470 $\pm$ 0.0181	0.9250 $\pm$ 0.0152	0.8315 $\pm$ 0.0549	Learnable
PROTEINS + GraphResGNN + SAGEConv	0.7178 $\pm$ 0.0411	0.7241 $\pm$ 0.0407	0.7151 $\pm$ 0.0397	0.7214 $\pm$ 0.0318	0.7160 $\pm$ 0.0248	0.7259 $\pm$ 0.0475	0.7088 $\pm$ 0.0521	Sparse 0.05
AIDS + NodeCrossGNN + GATConv	0.8970 $\pm$ 0.0491	0.9160 $\pm$ 0.0128	0.8960 $\pm$ 0.0483	0.8960 $\pm$ 0.0489	0.9060 $\pm$ 0.0268	0.9180 $\pm$ 0.0162	0.9025 $\pm$ 0.0139	Sparse 0.05
DD + NodeCrossGNN + GATConv	0.7080 $\pm$ 0.0172	0.7062 $\pm$ 0.0249	0.7029 $\pm$ 0.0160	0.7164 $\pm$ 0.0248	0.7147 $\pm$ 0.0259	0.7181 $\pm$ 0.0196	0.7105 $\pm$ 0.0243	Sparse 0.05

Table 16. Family-wise routing winners on representative targets.

Target	Best learnable	Best top-k	Best sparse	Overall winner
AIDS + GraphResGNN + GINConv	0.9500	0.9330	0.9470	Learnable
PROTEINS + GraphResGNN + SAGEConv	0.7178	0.7241	0.7259	Sparse
AIDS + NodeCrossGNN + GATConv	0.8970	0.9160	0.9180	Sparse
DD + NodeCrossGNN + GATConv	0.7080	0.7164	0.7181	Sparse

- [18] Ronghui You, Shuwei Yao, Hiroshi Mamitsuka, and Shanfeng Zhu. Deepgraphgo: graph neural network for large-scale, multispecies protein function prediction. *Bioinformatics*, 37(Suppl. 1):i262–i271, 2021.
- [19] Lingxiao Zhao and Leman Akoglu. Pairnorm: Tackling over-smoothing in gns. *arXiv preprint arXiv:2009.10698*, 2020.